

Improving CNN Performance for Multi-Class Skin Lesion Classification with Hyperparameter Tuning and Transfer Learning

Deep Learning Assignment Group 3
Data Science & Society
Tilburg University
03-10-2025

Elena Harutunyan (2050150): Hyperparameter Tuning, Methodology and Limitations
Noam Ladani (2121496): Baseline Implementation, Functions to plot results, Conclusion
Georgios Papadopoulos (2132530): Hyperparameter Tuning, Limitations and Future Improvements
Joachiem Swart (2126806): New Model Implementation, Transfer Learning, Discussion
Sylvana Wacanno (2002213): New Model Implementation, Transfer Learning
Jennicka Wijma (2127574): Baseline Implementation & Section, Functions to plot results, Discussion

Introduction

Skin is one of the largest and most important organs of the human body. Skin cancer is a serious disease that requires early and accurate diagnosis for successful treatment (Loescher et al., 2013). This assignment uses a dataset containing images of eight types of skin lesions, including three malignant cancers (BCC, MEL, and SCC). The aim is to develop a Deep Learning (DL) model capable of accurately detecting and classifying these lesions. DL enables Neural Networks (NN) to identify patterns and make predictions from large datasets (Pillai et al., 2024). A Convolutional Neural Network (CNN) was first developed as a baseline model for lesion classification and evaluated using metrics such as ROC/AUC, confusion matrices and precision/recall/F1. Model performance was then improved through six experiments involving hyperparameter tuning. Finally, a transfer learning approach using the DenseNet121 architecture was implemented and compared to the baseline- and improved models. The report concludes with a discussion on possible improvements, risks, and ethical considerations, with justifications for the chosen methods.

Baseline and Hyperparameter Tuning

The provided preprocessing pipeline was followed. The images were all resized to a 42x42 resolution with a batch size of 128, and split into training, validation and test sets. No additional preprocessing or data augmentation was applied. As instructed, the baseline model consists of two convolutional blocks, each followed by max-pooling, and two dense layers. The ReLu activation function was used for the hidden layers, while the Softmax activation was used in the output layer to predict class probabilities. The model is trained using categorical cross-entropy loss as this task involves multi-class classification where each image belongs to exactly one of the eight possible lesion categories. It was optimized with the Adam optimizer and trained for 15 epochs. The results are shown in Fig 1.

From this point, the hyperparameter tuning process began. We opted for tuning each hyperparameter incrementally, and comparing the performance to the baseline, which was a computational and time intensive process. Each chosen hyperparameter was carefully selected with the aim of improving performance. Six different hyperparameters were chosen for the tuning process and to improve the baseline model. An explanation of the six hyperparameters and their chosen values are explained below.

- **Dense Layer Depth**: An extra fully connected layer was added that could possibly increase the model's architecture to learn more complex decision boundaries and extract features. However, it failed as it did not improve the performance of the baseline.
- **Convolutional Capacity**: Two additional convolutional blocks were added to increase depth, allowing the network to capture "high-level patterns" and potentially improve generalization.
- **Number of Units**: The filter sizes of the four blocks were increased (32/64/128/256). This pattern is not arbitrary since it's been shown to balance capacity and generalization (Izquierdo-Cordova and Mayol-Cuevas, 2021).

- **Dropout**: After increasing the filter sizes, training accuracy increased to 75%, while validation stayed the same. A big sign of using regularization techniques came into play such as dropout and L2. These were chosen to mitigate overfitting by making it more robust, testing 4 different cases.
- **L2 Regularization**: L2 with λ of 0.0001 was applied to penalize large weights, encouraging smaller weight values and smoother decision boundaries.
- **Optimizer and Learning Rate**: The optimizer and learning rate were tuned, comparing the baseline Adam optimizer with AdamW, SGD (with Momentum) and RMSprop, with learning rates 0.00025, 0.0005 and 0.00075. These optimizers and learning rates were tested to potentially see improved stability and validation performance.

The results of the hyperparameter tuning and baseline performances are summarized in Table 1 with each column representing a hyperparameter configuration, and its validation accuracy and Macro AUC. Moreover, in Fig 2 the ROC/AUC and metrics of the final tuned model is shown.

Model ID	Baseline	Dense Layers	Conv. Layers	Number of Units	Dropout (0.5)	L2	Learning (0.0075)	Optimizer (rmsprop)
Val Accuracy	63.96%	64%	62.08%	64.47%	64.74%	64.23%	65.19%	65.32%
Val Macro AUC	86%	86%	85%	88%	87%	86%	88%	87%

Table 1 - Overview of the Performance Metrics of the Hyperparameters and Baseline.

Transfer Learning

To select the optimal transfer-learning model, we systematically compared multiple models. We first evaluated three different architectures (VGG16, ResNet50 and DenseNet121). Among the baseline configurations, DenseNet121 achieved the highest validation accuracy (as shown in Table 2) and was therefore selected to expand on.

VGG16	**ResNet50**	**DenseNet121**
60.89%	54.41%	64.09%

Table 2 - Validation accuracies for different architectures

After freezing the layers up until the fully connected layers, we experimented with adding dense layers and their setup. Firstly, we increased the units of the dense layer from 128 (baseline) to 256 units. However, this did not result in an improvement of the model. Secondly, more dense layers were added to the model. This did improve the model, albeit slightly. Next, we evaluated the dropout rate and L2 regularization. The dropout reduced the gap between training and validation accuracy, indicating reduced overfitting. By contrast, L2 regularization showed minimal impact. Next, we decreased the learning rate while training for 100 epochs and introducing early stopping monitoring validation loss to prevent overfitting. Again, we observed

no improvements. The final transfer learning model achieved a validation accuracy of 65.19%. The results of the model are shown in Fig 2.

Discussion

The final tuned model incorporated the optimized hyperparameters summarized in Table 3, resulting in improved validation and test accuracy compared to the baseline model. Increasing the number of convolutional layers from two to four allowed the network to capture more detail, while dropout and L2 regularization reduced overfitting, leading to better overall performance.

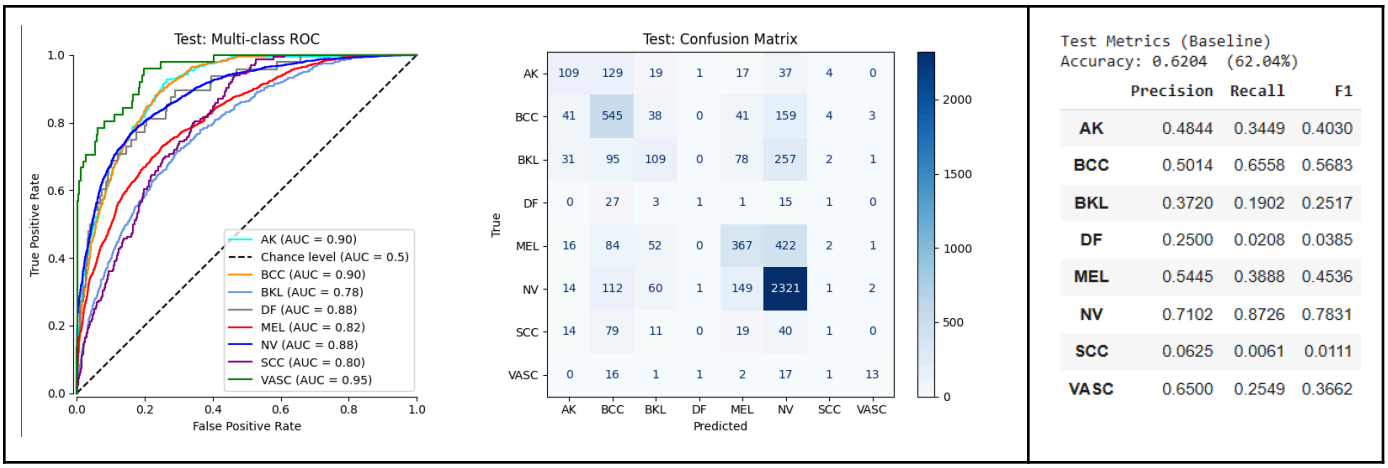
As shown in Figure 1, the baseline model achieved a test accuracy of 62.04% and a Macro AUC of 86%. Although its ROC curves indicated reasonable separability for some classes (BCC, AK, VASC), the confusion matrix revealed that others (DF, SCC) were frequently misclassified.

The tuned model (Figure 2) improved to a test accuracy of 63.70% and a Macro AUC of 87%, demonstrating stronger generalization and higher precision–recall values for BCC and NV. Despite this improvement, several minority classes continued to show low recall, suggesting class imbalance and limited representation in the dataset.

The transfer learning model based on DenseNet121 (Figure 3) achieved the highest overall test accuracy of 64.06% and strong per-class AUC values (AK=0.92, BCC=0.91, VASC=0.92). The pretrained feature extraction improved performance on several classes but the result is close to the tuned model, suggesting that DenseNet121 did not fully generalize to this specific dataset.

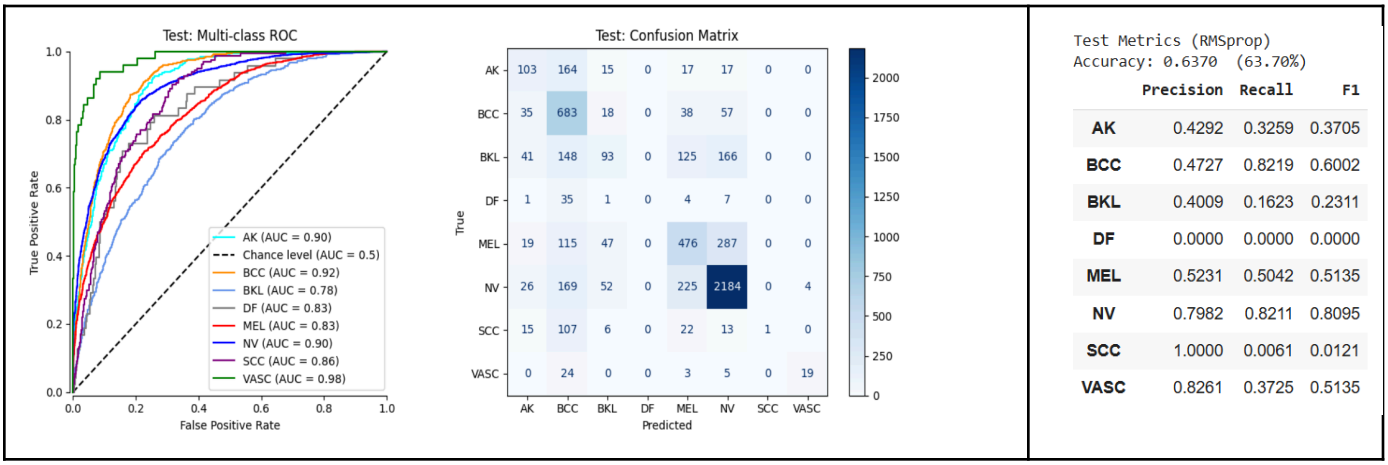
Hyperparameters	**Values**
Dense Layer Depth	1 hidden fully-connected layer
Convolutional Capacity	4 convolutional blocks
Number of Units	32,64,128,256
Dropout	0.50
L2 Regularization	$\lambda = 0.0001$
Optimizer and Learning Rate	RMSprop with learning rate 0.00075

Table 3 - Overview of the tuned hyperparameters in the final model



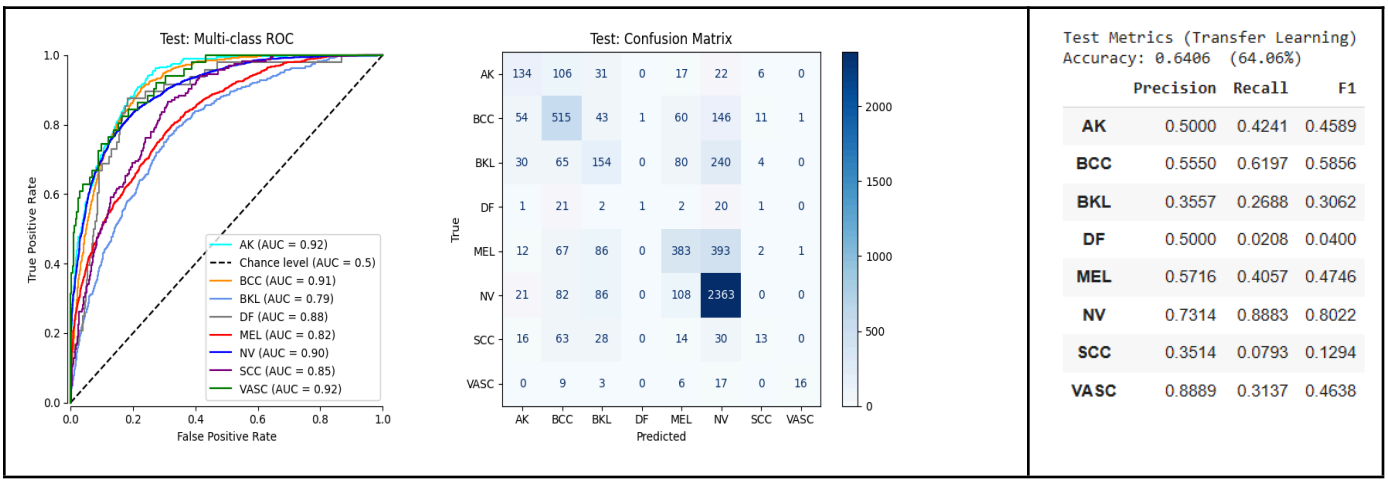
	Precision	Recall	F1
AK	0.4844	0.3449	0.4030
BCC	0.5014	0.6558	0.5683
BKL	0.3720	0.1902	0.2517
DF	0.2500	0.0208	0.0385
MEL	0.5445	0.3888	0.4536
NV	0.7102	0.8726	0.7831
SCC	0.0625	0.0061	0.0111
VASC	0.6500	0.2549	0.3662

Fig 1 - ROC/AUC, Confusion Matrix and Evaluation Metrics of the Baseline Model



	Precision	Recall	F1
AK	0.4292	0.3259	0.3705
BCC	0.4727	0.8219	0.6002
BKL	0.4009	0.1623	0.2311
DF	0.0000	0.0000	0.0000
MEL	0.5231	0.5042	0.5135
NV	0.7982	0.8211	0.8095
SCC	1.0000	0.0061	0.0121
VASC	0.8261	0.3725	0.5135

Fig 2 - ROC/AUC, Confusion Matrix and Evaluation Metrics of the final Tuned Model



	Precision	Recall	F1
AK	0.5000	0.4241	0.4589
BCC	0.5550	0.6197	0.5856
BKL	0.3557	0.2688	0.3062
DF	0.5000	0.0208	0.0400
MEL	0.5716	0.4057	0.4746
NV	0.7314	0.8883	0.8022
SCC	0.3514	0.0793	0.1294
VASC	0.8889	0.3137	0.4638

Fig 3 - ROC/AUC, Confusion Matrix and evaluation metrics of the final Transfer Learning model

Conclusion

To conclude, we first implemented a CNN baseline model for skin lesion classification and evaluated its performance using performance metrics, confusion matrices, and ROC/AUC

curves. We then tweaked hyperparameters, like adding dense and convolutional layers, increasing filters per layer, using dropout, L2 regularization and experimenting with optimizers, to try to boost its performance. However, the improvements were limited due to constraints we faced that further held back results.

Lastly, we implemented a new model using transfer learning by leveraging the DenseNet121 architecture for feature extraction. After freezing the layers up to the fully connected layers and experimenting with the dense layer setup, no significant performance improvement was observed, in comparison to the tuned CNN. This indicates that the features learned from DenseNet121 do not generalize well to the skin lesion domain.

Limitations of Current Work

A limitation of this study is that the preprocessing pipeline was provided, meaning the image resolution (42×42) could not be adjusted. The relatively low resolution restricted the level of detail available for the model to learn from, making improvements through hyperparameter tuning more challenging. Another key limitation was the time and hardware constraints, which limited the number of experiments and hyperparameter configurations that could be tested. Finally, the dataset was highly imbalanced which was a big problem for the model to distinguish the minority classes. All the above likely restricted the model's ability to reach its full performance potential.

Suggestions for Future Research

Based on existing literature, better hyperparameter optimization techniques could be employed, such as Bayesian optimization or GridSearchCV to more efficiently search the hyperparameter space (Onorato et al. 2024; Singh et al., 2023). For example, Zhang et al. (2023) propose a dynamic Bayesian method with resource-aware balancing for HPO under constrained resources SpringerOpen. Multi-fidelity Bayesian optimization is also promising, by using cheaper proxies (e.g., fewer epochs or subsets of data) to accelerate the search (Wu et al. 2019). Techniques to overcome the imbalanced problem such as k-fold cross validation and class weights could be promising to have better generalization (Uzer, 2025). Also, further preprocessing such an increase of the resolution to 256 by 256 where most of the CNN models are trained on together with different batch sizes. Beyond technical improvements, several ethical and societal aspects must be considered:

Bias and Fairness: The dataset may not equally represent all skin tones, age groups, or lesion types. This could lead to biased predictions, where the algorithm performs well on certain populations but poorly on underrepresented groups, potentially causing health disparities.

Privacy Concerns: Medical image data is highly sensitive. If not handled with strict data protection standards, there is a risk of patient data misuse, leaks, or unauthorized access.

Security Vulnerabilities: Malicious actors could manipulate or tamper with the system (e.g., adversarial attacks on images), leading to incorrect diagnoses. Ensuring secure model deployment, encryption, and robust monitoring is essential.

Societal Impact: Over-reliance on automated systems could lead to misdiagnoses if the algorithm is used without medical supervision. While the tool may aid dermatologists, it should not replace professional judgment. Incorrect predictions can cause either unnecessary anxiety (false positives) or delayed treatment for serious conditions (false negatives).

Accountability and Transparency: Deep learning models are often “black boxes”. A lack of interpretability may make it difficult for clinicians to trust or explain the algorithm’s decisions to patients. This raises accountability issues if harm occurs due to algorithmic errors.

Bibliography

Izquierdo-Cordova, R., & Mayol-Cuevas, W. (2021). Filter distribution templates in convolutional networks for image classification tasks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2104.13993

Loescher, L. J., Janda, M., Soyer, H. P., Shea, K., & Curiel-Lewandrowski, C. (2013). Advances in skin cancer early detection and diagnosis. *Seminars in Oncology Nursing*, *29*(3), 170–181. https://doi.org/10.1016/j.soncn.2013.06.003

Onorato, G. (2024). Bayesian Optimization for Hyperparameters Tuning in Neural Networks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2410.21886

Pillai, R., Sharma, N., Gupta, S., Gupta, D., Juneja, S., Malik, S., Qin, H., Alqahtani, M. S., & Ksibi, A. (2024). Enhanced skin cancer diagnosis through grid search algorithm-optimized deep learning models for skin lesion analysis. *Frontiers in Medicine*, *11*. https://doi.org/10.3389/fmed.2024.1436470

Singh, J., Sandhu, J. K., & Kumar, Y. (2023). An Analysis of Detection and Diagnosis of Different Classes of Skin Diseases Using Artificial Intelligence-Based Learning Approaches with Hyper Parameters. *Archives of Computational Methods in Engineering*, *31*(2), 1051–1078. https://doi.org/10.1007/s11831-023-10005-2

Wu, J., Toscano-Palmerin, S., Frazier, P. I., & Wilson, A. G. (2019). Practical multi-fidelity Bayesian optimization for hyperparameter tuning. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.1903.04703

Zhang, J., Wang, T., Wang, B., Chen, C., & Wang, G. (2023). Hyperparameter optimization method based on dynamic Bayesian with sliding balance mechanism in neural network for cloud computing. *Journal of Cloud Computing Advances Systems and Applications*, *12*(1). https://doi.org/10.1186/s13677-023-00482-y